How the orders service serves a read

A read of one order, `GET /orders/42`, is answered from Redis when it can be and from Postgres when it cannot. This note shows the path, what a miss costs, and where the time goes.

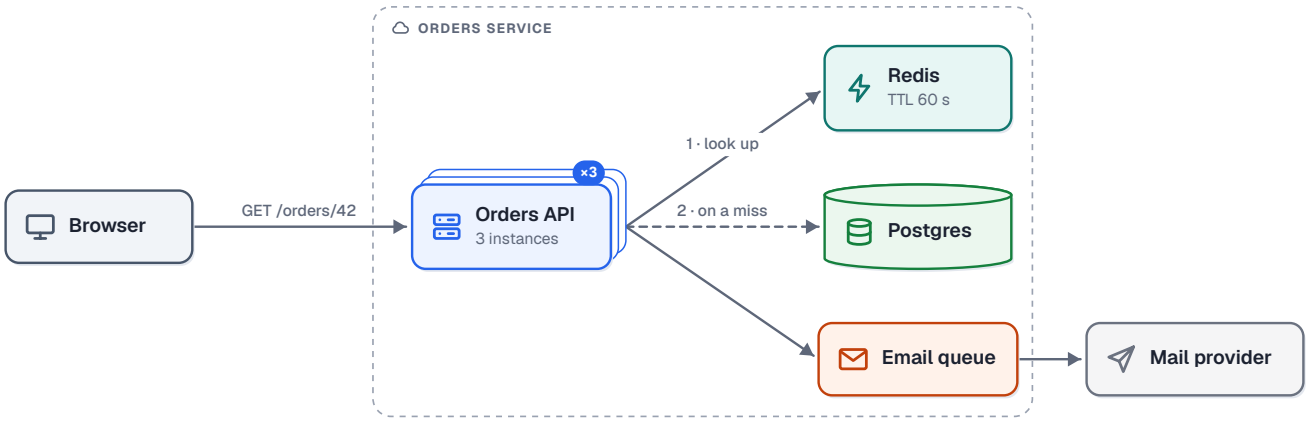

Figure 1: Three API instances behind one address; Postgres only on a miss (the dashed arrow).

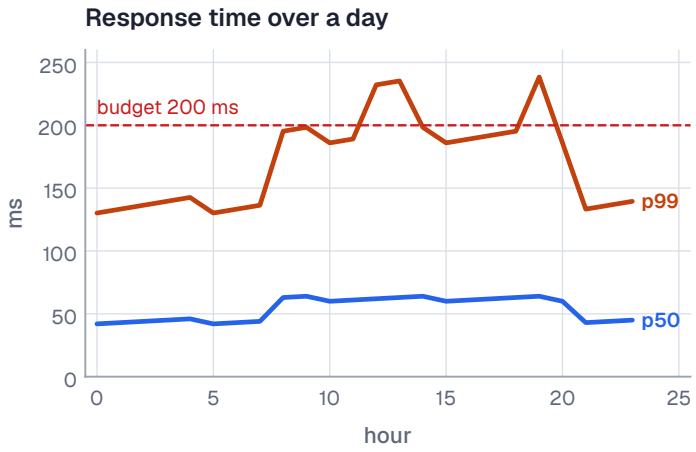

Figure 2: p99 crosses the budget three times a day.

Key idea

A miss is a slower read, not an error: the API fills the cache on its way back.

Read	p50	p99
Hit	9 ms	13 ms
Miss	41 ms	48 ms

```python
def get_order(order_id: int) -> Order:
    cached = redis.get(f"order:{order_id}")
    if cached is not None:              # a hit: 13 ms at p99
        return Order.parse(cached)
    return db.fetch_order(order_id)     # a miss: Postgres
```